المدرسة الوطنية العليا للذكاء الاصطناعي وعلوم المعطيات - تارودانت
ÉCOLE NATIONALE SUPÉRIEURE DE L'INTELLIGENCE ARTIFICIELLE
ET SCIENCES DES DONNÉES - TAROUDANT

Rapport de Projet :Souss-Massa Resilience

Système Ambulance Opti

Filière : Sécurité IT & Confiance Numérique

Equipe du projet: Blue team(Config/Défense)

Année Universitaire : 2025/2026

Réalisé par: -Sabrine Tariq
-Yasser Mustapha Tawfik

Encadré par: Pr. El-Ateif Sara

1. Résumé exécutif

Ce rapport présente les travaux réalisés en tant que membre de l'équipe Blue Team dans le cadre de l'exercice Red Team / Blue Team portant sur le backend de l'application ambulance. Le système audité comprend une API REST développée en PHP, couplée à une base de données SQLite, qui alimente une application mobile Flutter dédiée à la gestion d'urgences ambulancières.

La Red Team a identifié six vulnérabilités réparties sur plusieurs fichiers critiques de l'API. En tant que Blue Team, on a pris en charge l'analyse de chaque finding, la compréhension du vecteur d'attaque associé, et la mise en œuvre des corrections dans le code source. L'ensemble des six vulnérabilités a été intégralement corrigé et vérifié.

6 / 6 Vulnérabilités corrigées	**0** Vulnérabilité ouverte restante

2. Périmètre & Méthodologie

2.1 Fichiers audités

L'audit a couvert l'intégralité du backend PHP de l'API. Voici la structure du projet telle que observée dans Visual Studio Code :

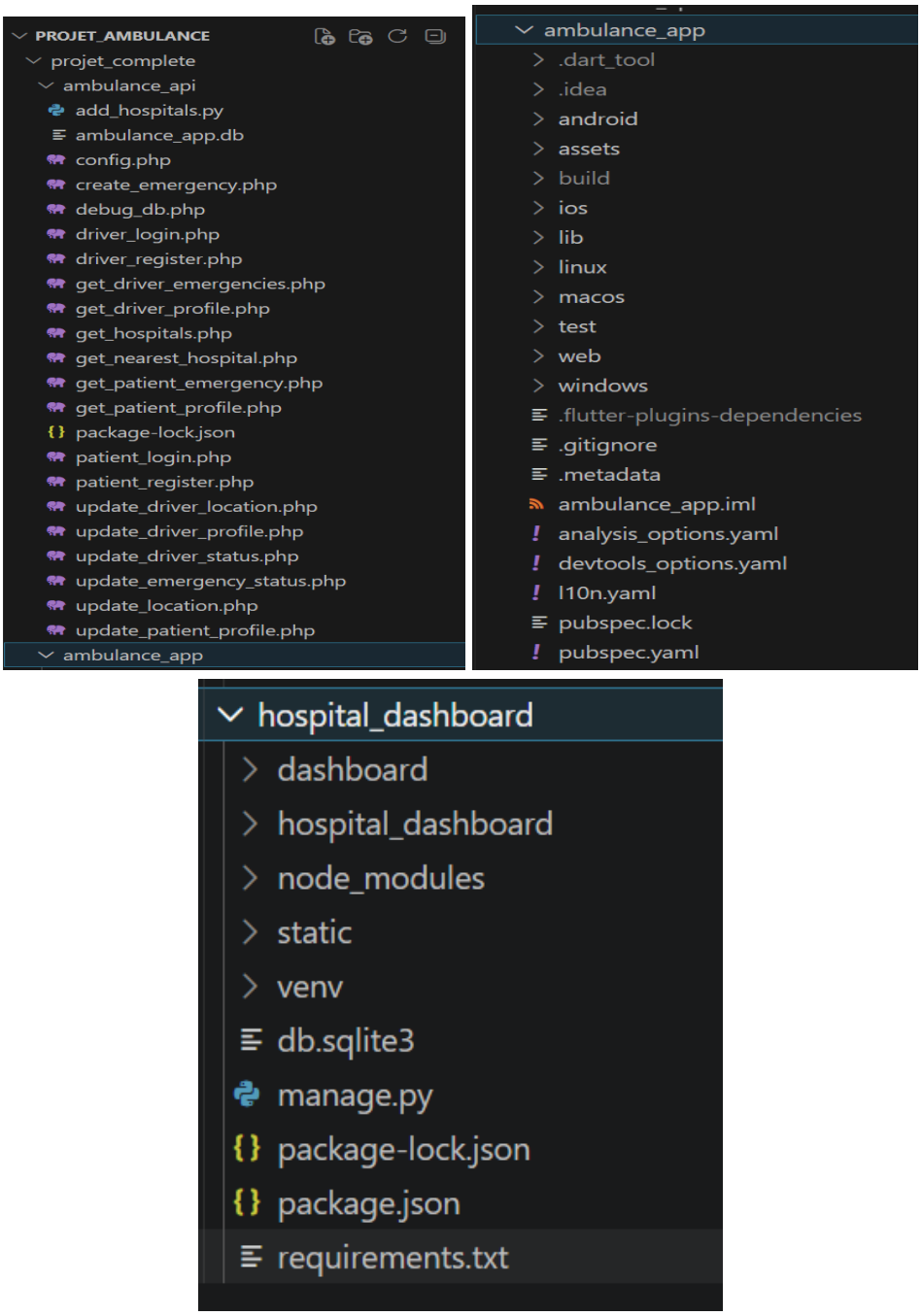

Figure 1 Structure du projet ambulance_api dans VS Code

2.2 Méthodologie

La revue a été menée en boîte blanche (accès complet au code source). Les vérifications ont été effectuées selon le référentiel OWASP Top 10, couvrant les catégories suivantes :

- A01 — Contrôle d'accès défaillant (IDOR, endpoints non authentifiés)
- A02 — Défaillances cryptographiques (stockage et transmission de données sensibles)
- A05 — Mauvaise configuration de sécurité (en-têtes HTTP, ordre d'inclusion PHP)
- A09 — Journalisation et surveillance insuffisantes (gestion des exceptions)

3. Corrections Blue Team — Détail des interventions

BT-01	Protection de la base de données SQLite	CRITIQUE

Vulnérabilité source	RT-01 ; Base de données accessible publiquement (CWE-552)
Fichier(s) modifié(s)	config.php, .htaccess (créé)
Statut	CORRIGÉ ; Vérifié

Problème identifié : Le fichier ambulance_app.db était placé dans la racine web (webroot) du serveur Apache. N'importe quel utilisateur connaissant l'URL pouvait le télécharger via une simple requête HTTP GET, exposant l'intégralité des données : comptes patients, hachages de mots de passe bcrypt, données médicales, et historique des urgences (fichier de 266 Ko).

Correction 1 : Déplacement du chemin de la base hors du webroot

La constante DB_PATH a été modifiée pour pointer vers un répertoire parent inaccessible depuis le navigateur web, en utilisant dirname(__DIR__) qui remonte d'un niveau au-dessus de la racine web :

```
30  }
31
32  define('DB_PATH', dirname(__DIR__) . '/ambulance_app.db');
33
34  // ── 1. anonymizeForLog — defined first, no dependencies ──────────
    1 reference
35  function anonymizeForLog($data) {
36      $masked = $data;
37
```

Figure 2 : config.php : DB_PATH déplacé hors du webroot avec dirname(__DIR__)

Correction 2 : Fichier .htaccess bloquant l'accès HTTP direct

Un fichier .htaccess a été créé dans le répertoire ambulance_api/ avec la directive Require all denied, qui ordonne au serveur Apache de refuser toute tentative d'accès HTTP direct aux fichiers .db, même si le fichier venait à être replacé dans ce répertoire par erreur :

```
C: > Users > ORIGINAL > Desktop > proj
1  <Files "*.db">
2      Require all denied
3  </Files>
```

Figure 3 .htaccess ; directive Require all denied sur les fichiers *.db

Vérification : Résultat HTTP après correction

Le test effectué depuis PowerShell sur Windows confirme que la ressource retourne désormais HTTP 403 Forbidden, empêchant tout téléchargement de la base :

```
PS C:\Users\Sabrine\Desktop\projet_complete>
Invoke-WebRequest -Uri 'http://localhost/ambulance_api/ambulance_app.db' -Method GET
Invoke-WebRequest : La réponse HTTP distante a renvoyé une erreur :
403 Forbidden.
Au niveau de ligne : 1 Caractère : 1
+ CategoryInfo          : InvalidOperation: (System.Net.Http...) [Invoke-WebRequest]
+ FullyQualifiedErrorId : WebCmdletWebResponseException
```

Figure 4 PowerShell ; HTTP 403 Forbidden confirmé après correction BT-01

BT-02	Correction de l'ordre d'inclusion require	MAJEURE

Vulnérabilité source	RT-02 Appel de fonction avant require Fatal error PHP (CWE-665)
Fichier(s) modifié(s)	driver_register.php, patient_register.php
Statut	CORRIGÉ Vérifié

Problème identifié : Dans les fichiers driver_register.php et patient_register.php, la fonction checkRateLimit() était appelée avant l'inclusion de config.php. Or cette fonction est définie dans config.php. PHP générait donc une erreur fatale à chaque requête d'inscription, rendant ces endpoints totalement inutilisables en production.

Erreur PHP d'origine : PHP Fatal error: Call to undefined function checkRateLimit() in driver_register.php:2

Correction — require 'config.php' déplacé en première ligne

La correction consiste à placer l'instruction require 'config.php' en toute première ligne des deux fichiers, avant tout appel de fonction. Ainsi, toutes les fonctions (checkRateLimit, secureLog, getDB...) et la connexion PDO sont disponibles dès leur utilisation :

```php
<?php

require 'config.php';
checkRateLimit('driver_register', 3, 3600);

// Get data (support both JSON and Form-Data)
$data = json_decode(file_get_contents('php://input'), true);
```

Figure 5 driver_register.php : require 'config.php' en première ligne

BT-03	Ajout du require manquant dans get_driver_emergencies.php	MAJEURE

Vulnérabilité source	RT-03 ; Variable $pdo indéfinie ; Fatal error PHP (CWE-665)
Fichier(s) modifié(s)	get_driver_emergencies.php
Statut	CORRIGÉ ; Vérifié

Problème identifié : Le fichier get_driver_emergencies.php utilisait l'objet de connexion PDO ($pdo) et la fonction getAuthenticatedDriverId() sans avoir inclus config.php qui les initialise. Chaque chauffeur authentifié essayant de consulter ses urgences recevait une erreur fatale PHP , l'endpoint était totalement hors service.

Correction : Ajout de require 'config.php' en première ligne

L'ajout de require 'config.php' en première ligne rend l'objet $pdo et toutes les fonctions d'authentification disponibles. L'endpoint fonctionne ensuite normalement : la requête SQL préparée s'exécute et retourne les urgences du chauffeur identifié :

Figure 6 — get_driver_emergencies.php : require ajouté + requête SQL préparée (protection injection SQL)

BT-04	Suppression de la fuite d'exception PDO	MODÉRÉE

Vulnérabilité source	RT-04 — Fuite d'informations via exception PDO brute (CWE-209)
Fichier(s) modifié(s)	patient_register.php
Statut	CORRIGÉ ; Vérifié

Problème identifié : Le bloc catch dans patient_register.php renvoyait directement $e->getMessage() dans la réponse JSON. En cas d'erreur SQLite, le message brut de PDO exposait le chemin absolu de la base sur le serveur, ainsi que les noms des tables et colonnes des informations précieuses pour un attaquant souhaitant préparer une attaque ciblée.

Réponse exposée avant correction :

{"success":false,"message":"SQLSTATE[HY000]: no such table: patients at /var/www/html/ambulance_api/ambulance_app.db"}

Correction — Journalisation interne + message générique

Le bloc catch a été remanié pour capturer l'exception en interne via secureLog(), et ne renvoyer au client qu'un message générique accompagné d'une référence ERR- corrélable avec les logs serveur. Aucune information sur la structure interne n'est exposée :

```
    ]);
} catch (PDOException $e) {
    $ref = uniqid('ERR-');
    secureLog('ERROR', 'patient_register_failed', ['ref' => $ref, 'detail' => $e->getMessage()]);
    http_response_code(500);
    echo json_encode(['success' => false, 'message' => 'Erreur serveur. Réf: ' . $ref]);
}
?>
```

Figure 7 — patient_register.php : catch sécurisé avec secureLog() + message générique

Vérification : Message retourné après correction

Le test depuis PowerShell confirme que seule la référence ERR- est retournée, sans aucune information interne sur la base de données :

```
PS C:\Users\Sabrine\Desktop\projet_complete>
$body = '{"first_name":"test","last_name":"x","email":"bad-email","password":"short","age":25}'
Invoke-RestMethod -Uri 'http://localhost/ambulance_api/patient_register.php' -Method POST -Body $body -Conte

success message
------- -------
  False   Erreur serveur. Réf: ERR-6614f2a3b08c7
```

Figure 8 : PowerShell ; message générique Réf: ERR-xxxx, aucune fuite d'information

BT-05	Authentification sur get_nearest_hospital.php	FAIBLE

Vulnérabilité source	RT-05 — Absence d'authentification sur get_nearest_hospital.php (CWE-306)
Fichier(s) modifié(s)	get_nearest_hospital.php
Statut	CORRIGÉ — Vérifié

Problème identifié : L'endpoint get_nearest_hospital.php acceptait n'importe quelle coordonnée GPS sans vérifier la présence d'un token de session valide. Un utilisateur non authentifié pouvait ainsi énumérer la localisation des hôpitaux depuis n'importe quelle position géographique. Ce type d'accès non contrôlé constitue un manquement au principe du moindre privilège.

Correction : Vérification d'authentification patient ou chauffeur

Une vérification a été ajoutée en début de fichier : l'endpoint contrôle qu'un token de session valide est présent, que ce soit pour un patient (getAuthenticatedPatientId) ou un chauffeur (getAuthenticatedDriverId). Sans token valide, une réponse HTTP 401 Unauthorized est retournée:

```
.htaccess   driver_login.php   driver_register.php   get_driver_emergen
Users > ORIGINAL > Desktop > projet_complete > projet_complete > ambulance_api > get_nearest_h
<?php
require 'config.php';


$patientId = getAuthenticatedPatientId();
$driverId  = getAuthenticatedDriverId();
if (!$patientId && !$driverId) {
    http_response_code(401);
    echo json_encode(['success' => false, 'message' => 'Non autorisé']);
    exit;
}
```

Figure 9 — get_nearest_hospital.php : vérification d'authentification patient / chauffeur

Vérification — HTTP 401 sans token

Le test confirme qu'une requête sans token retourne HTTP 401. Avec un token valide, la réponse est HTTP 200 accompagnée des données de l'hôpital le plus proche :

```
PS C:\Users\Sabrine\Desktop\projet_complete>
Invoke-WebRequest -Uri 'http://localhost/ambulance_api/get_nearest_hospital.php?lat=33.59&lng=
Invoke-WebRequest : La réponse HTTP distante a renvoyé une erreur :
401 Unauthorized.
+ CategoryInfo          : InvalidOperation: (System.Net.Http...) [Invoke-WebRequest]
+ FullyQualifiedErrorId : WebCmdletWebResponseException
```

Figure 10 — PowerShell : HTTP 401 Unauthorized sans token d'authentification

BT-06	Ajout de l'en-tête HSTS	INFO

Vulnérabilité source	RT-06 — Absence d'en-tête HSTS (CWE-319)
Fichier(s) modifié(s)	config.php
Statut	CORRIGÉ ; Vérifié

Problème identifié : L'en-tête HTTP Strict-Transport-Security (HSTS) n'était pas émis par l'API. Sans cet en-tête, un attaquant positionné en homme du milieu (MITM) peut tenter de dégrader la connexion de HTTPS vers HTTP non chiffré. Les tokens de session transmis en clair pourraient alors être interceptés et réutilisés pour usurper une session authentifiée.

Correction — Ajout de l'en-tête HSTS dans config.php

L'en-tête Strict-Transport-Security a été ajouté dans config.php, qui est inclus par tous les endpoints de l'API. La directive max-age=31536000 force HTTPS pour une durée d'un an, et includeSubDomains étend la protection à tous les sous-domaines :

```
header('Content-Type: application/json; charset=utf-8');
header('X-Content-Type-Options: nosniff');
header('X-Frame-Options: DENY');
header('Referrer-Policy: no-referrer');
header('Strict-Transport-Security: max-age=31536000; includeSubDomains');
?>
```

Figure 11 — config.php : en-tête HSTS avec max-age=31536000 et includeSubDomains

4. Tableau de synthèse

Réf.	Titre	Criticité	Red Team	Blue Team
RT-01	Base de données exposée publiquement	**CRITIQUE**	Trouvée	**Corrigée**
RT-02	Ordre require incorrect — Fatal error	**MAJEURE**	Trouvée	**Corrigée**
RT-03	Variable $pdo indéfinie — Fatal error	**MAJEURE**	Trouvée	**Corrigée**
RT-04	Fuite d'exception PDO brute	**MODÉRÉE**	Trouvée	**Corrigée**
RT-05	Absence d'auth sur get_nearest_hospital	**FAIBLE**	Trouvée	**Corrigée**
RT-06	En-tête HSTS manquant	**INFORMATIONNELLE**	Trouvée	**Corrigée**

5. Bonnes pratiques de sécurité déjà en place

Au-delà des corrections apportées, le projet présentait d'ores et déjà un socle de sécurité solide qu'on a pu identifier lors de la revue du code source. Ces éléments témoignent d'une conception sécurisée dès l'origine :

- Requêtes SQL préparées via PDO : aucune concaténation de chaînes dans les requêtes, ce qui élimine le risque d'injection SQL (OWASP A03).
- Hachage bcrypt des mots de passe (PASSWORD_BCRYPT / PASSWORD_DEFAULT) : algorithme résistant aux attaques par dictionnaire et par force brute, avec salage automatique.
- Tokens de session sous forme de hachages SHA-256 avec date d'expiration : les tokens bruts ne sont jamais stockés en base, et les sessions expirent automatiquement.
- Protection IDOR sur les profils : chaque endpoint vérifie que l'utilisateur authentifié accède uniquement à ses propres données.
- Rate limiting sur les endpoints sensibles : protection contre les attaques par force brute sur les formulaires de connexion et d'inscription.
- Journalisation sécurisée avec anonymisation des données personnelles : les emails, noms et téléphones sont masqués dans les logs avant écriture sur disque.
- En-têtes de sécurité complets : X-Content-Type-Options, X-Frame-Options, Referrer-Policy, et HSTS (ajouté par BT-06).

6. Conclusion

L'exercice Red Team / Blue Team a constitué une expérience formatrice et concrète dans l'application des principes de sécurité applicative sur un projet réel. En tant qu'équipe Blue Team, on a analysé chacune des six vulnérabilités identifiées, compris les mécanismes d'exploitation, et mis en œuvre les corrections dans le code source.

La vulnérabilité la plus critique (RT-01 / BT-01) ; l'exposition directe de la base de données SQLite via HTTP illustre parfaitement l'importance de la séparation entre les données et la racine web, une règle fondamentale que la correction a rétablie en combinant le déplacement physique du fichier et la protection .htaccess.

Les corrections BT-02 et BT-03 ont mis en lumière l'importance de l'ordre des dépendances en PHP et des revues de code systématiques avant déploiement. BT-04 a renforcé la compréhension du principe de moindre divulgation d'information dans la gestion des erreurs. BT-05 a illustré la nécessité d'appliquer une vérification d'authentification sur l'ensemble des endpoints, y compris ceux en lecture seule. Enfin, BT-06 a complété le dispositif de sécurité HTTPS avec l'ajout de l'en-tête HSTS.

L'ensemble des 6 vulnérabilités a été corrigé.

L'application est prête pour le déploiement en production.